



MICRO SERVIÇOS E SISTEMAS ESCALÁVEIS

Aula 2 — Sistemas Distribuídos na Prática

Aprofundando os fundamentos e dando os primeiros passos reais com microserviços

Prof. Renan de Carvalho Oliveira

Curso de Sistemas de Informação · 7º período · Disciplina SIS440



Hoje tem código:
Mini iFood com 2 microserviços

Duração: 2 horas-aula

Onde paramos



O "inferno monolítico"

Complexidade, deploys arriscados e escala travada quando a aplicação cresce demais.



Microserviços, em uma frase

Decompor a aplicação em serviços pequenos, cada um com API e dados próprios.



Scale Cube

Eixo X (clonar), Eixo Z (particionar), Eixo Y (decompor por funcionalidade).



Hoje: aprofundamos os 4 pilares de sistemas distribuídos e já colocamos a mão no código, construindo os dois primeiros microserviços do projeto do semestre.

O que vamos ver hoje

01

Transparência

Os diferentes tipos e onde aparecem no dia a dia

03

Openness

Por que padrões abertos facilitam (ou travam) a evolução

05

Falácias distribuídas

As 8 suposições que todo dev iniciante faz — e erra

02

Escalabilidade

Dimensões e técnicas para crescer sem redesenhar tudo

04

Confiabilidade

Disponibilidade x confiabilidade e falhas parciais

06

Mão na massa

Criando pedido-service e pagamento-service em Java + Maven

Transparência tem vários "sabores"

Cada tipo esconde um aspecto diferente da natureza distribuída do sistema:



De Acesso

O jeito de acessar um recurso local ou remoto é o mesmo (a mesma chamada REST, não importa onde o serviço está).



De Localização

Quem usa não precisa saber em qual máquina/servidor o recurso está fisicamente.



De Replicação

Múltiplas cópias de um dado existem, mas o usuário enxerga uma coisa só.



De Concorrência

Vários usuários acessam o mesmo recurso ao mesmo tempo sem perceberem interferência uns dos outros.



De Migração

Um recurso pode ser movido de lugar sem que quem o usa perceba ou precise mudar código.



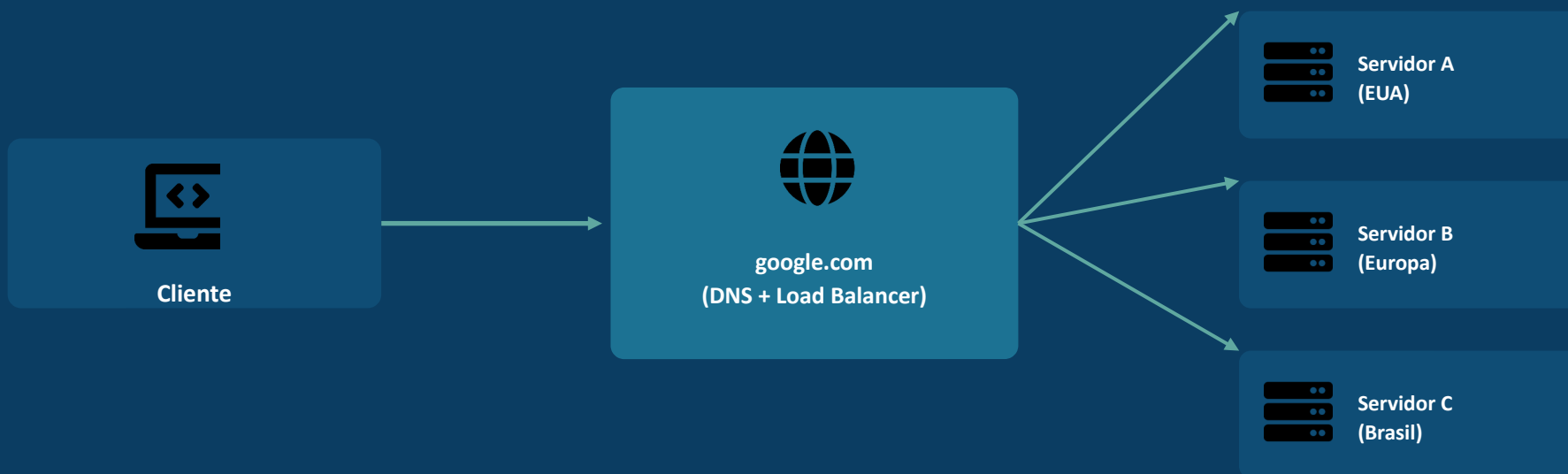
De Falha

Falhas e recuperações de componentes ficam escondidas de quem usa o sistema.

EXEMPLO PRÁTICO

Você já usou transparência de localização hoje

Quando seu navegador acessa google.com, você não sabe (nem precisa saber) em qual datacenter, país ou servidor físico a requisição foi realmente atendida.



O mesmo princípio vale para os microsserviços que vamos construir: quem chama a API não sabe (nem precisa saber) em que máquina o serviço está rodando.

Escalar em quais dimensões?

DIMENSÕES



Tamanho

Suportar mais usuários e mais requisições sem perda de desempenho.



Geográfica

Usuários e dados espalhados por regiões diferentes, com latência aceitável para todos.



Administrativa

Crescer mesmo quando o sistema passa a ser gerenciado por várias organizações/times diferentes.

TÉCNICAS



Replicação

Múltiplas cópias do mesmo dado/serviço para aumentar disponibilidade e desempenho.



Particionamento

Dividir os dados em fatias (ex.: por região), cada uma tratada por uma instância diferente.



Cache + esconder latência

Guardar resultados já calculados e disparar operações de forma assíncrona sempre que possível.

Abertura: a favor de quem vem depois

Um sistema aberto segue padrões públicos e bem documentados de interface — o que permite integrar, substituir e evoluir peças com previsibilidade.

PADRÃO ABERTO

REST + JSON sobre HTTP

- ✓ Documentado publicamente e amplamente adotado
- ✓ Qualquer linguagem/plataforma consegue consumir
- ✓ Fácil de testar (curl, Postman, navegador)
- ✓ Favorece a interoperabilidade entre serviços

PROTOCOLO PROPRIETÁRIO

Formato binário fechado de um fornecedor

- ✗ Só ferramentas do próprio fornecedor conseguem ler
- ✗ Difícil de integrar com serviços de terceiros
- ✗ Depuração exige ferramentas específicas e pagas
- ✗ Aumenta o "vendor lock-in" do sistema

Disponibilidade não é a mesma coisa que confiabilidade



Disponibilidade

A fração do tempo em que o sistema está no ar e respondendo. Um sistema pode estar "disponível" e ainda assim devolver respostas erradas.



Confiabilidade

A garantia de que o sistema faz exatamente o que deveria fazer, de forma consistente — mesmo diante de falhas parciais.



No nosso projeto de hoje: se o pedido-service estiver fora do ar, o pagamento-service falha ao tentar chamá-lo. Um sistema confiável precisa prever esse cenário — assunto do Módulo 5 (Tolerância a Falhas).

As 8 Falácias da Computação Distribuída

Peter Deutsch (Sun Microsystems) listou suposições que todo desenvolvedor iniciante faz sobre redes — e que sempre se provam falsas:

1

A rede é confiável

2

A latência é zero

3

A largura de banda é infinita

4

A rede é segura

5

A topologia não muda

6

Existe um administrador só

7

O custo de transporte é zero

8

A rede é homogênea

No pagamento-service de hoje, a chamada ao pedido-service passa pela rede de verdade — mesmo rodando no mesmo notebook. É o primeiro contato prático com essas falácias.



DA TEORIA À PRÁTICA

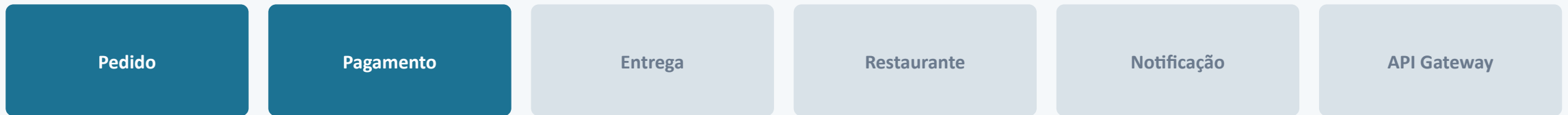
Vamos construir nosso primeiro sistema distribuído de verdade

Dois projetos Java + Maven, dois processos separados, se comunicando por HTTP — exatamente como discutimos até agora.

O projeto do semestre: Mini iFood

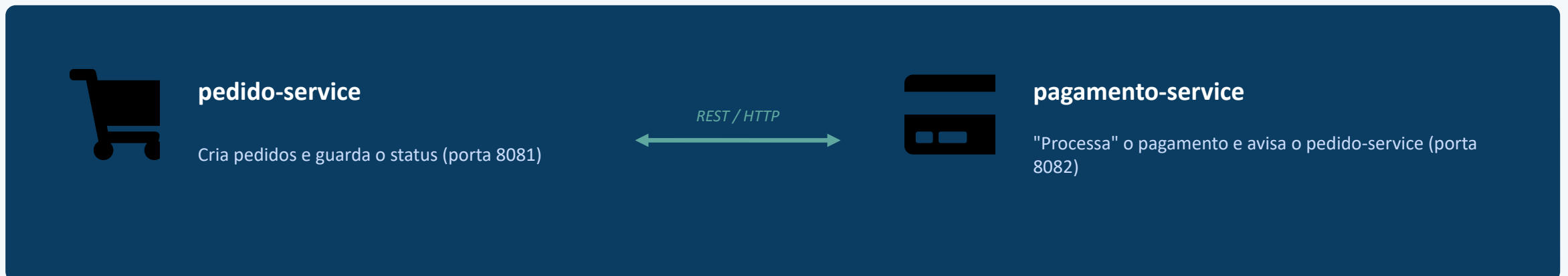
Vamos evoluir este sistema aula a aula, aplicando cada módulo do plano de ensino. A visão completa (futuro) e o ponto de partida (hoje):

VISÃO COMPLETA (AO LONGO DO SEMESTRE)

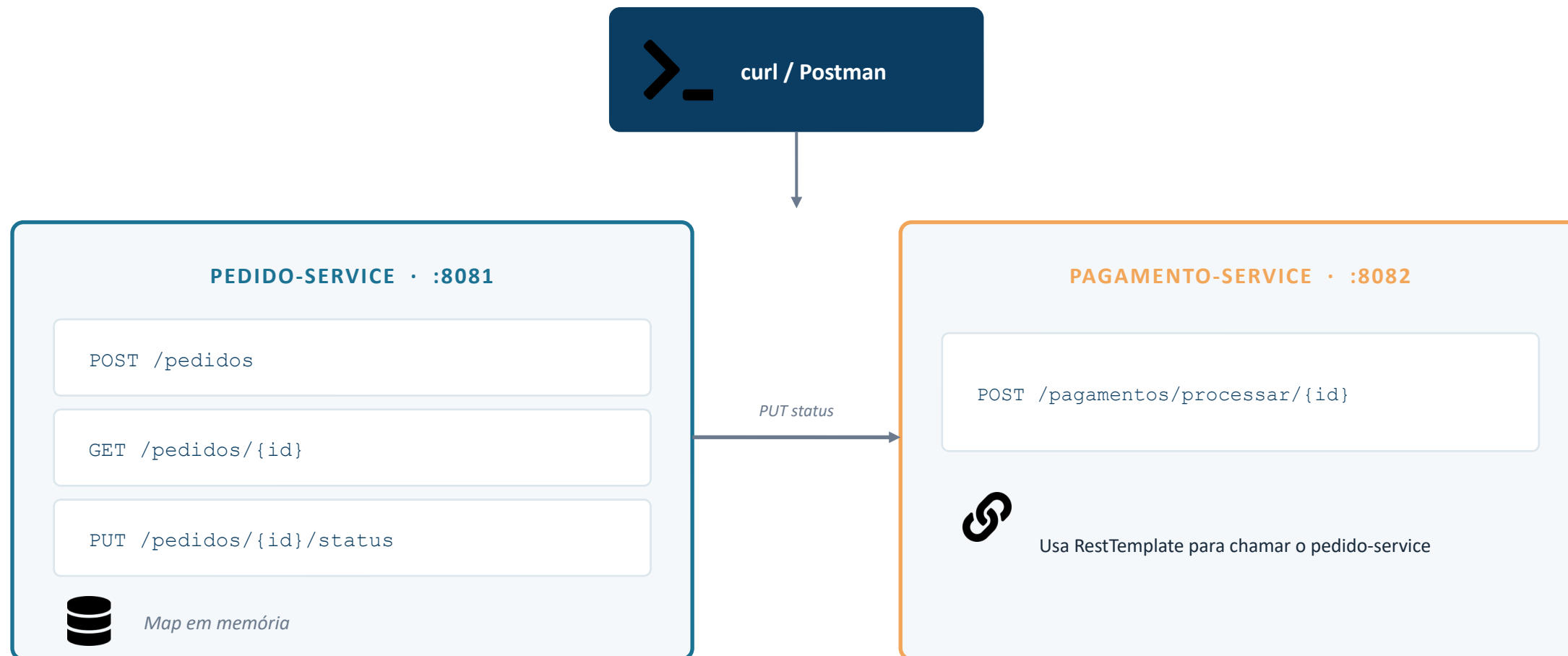


Em cinza: chegaremos lá nos próximos módulos (comunicação, tolerância a falhas, nuvem...).

PONTO DE PARTIDA (HOJE)



Arquitetura que vamos construir



Passo 1 — Criar os projetos Maven

1

Acessar start.spring.io

Project: Maven · Language: Java · Spring Boot 3.2.x · Packaging: Jar · Java 17

2

Adicionar a dependência

Spring Web (é só o que precisamos para criar uma API REST hoje)

3

Gerar e abrir no VS Code

Baixar o .zip, extrair e abrir a pasta com a extensão "Extension Pack for Java" instalada

4

Repetir para os dois serviços

Um projeto chamado pedido-service e outro pagamento-service, cada um em sua própria pasta

```
pedido-service/          pagamento-service/
├── pom.xml               ├── pom.xml
└── src/main/java/.../pedido/  └── src/main/java/.../pagamento/
```

Passo 2 — pedido-service: modelo e API

Pedido.java — o que é um pedido

```
public class Pedido {  
    private Long id;  
    private String cliente;  
    private String itens;  
    private BigDecimal valor;  
    private String status; // CRIADO, PAGO...  
    // getters e setters  
}
```

PedidoController.java — criar pedido

```
@RestController  
@RequestMapping("/pedidos")  
public class PedidoController {  
  
    @PostMapping  
    public ResponseEntity<Pedido>  
        criarPedido(@RequestBody Pedido p) {  
        // salva no Map, gera id, status=CRIADO  
    }  
}
```

PedidoController.java — atualizar status (o pagamento-service vai chamar este endpoint)

```
@PutMapping("/{id}/status")  
public ResponseEntity<Pedido> atualizarStatus(  
    @PathVariable Long id,  
    @RequestBody AtualizarStatusRequest request) {  
  
    Pedido pedido = pedidos.get(id);  
    pedido.setStatus(request.getStatus());  
    return ResponseEntity.ok(pedido);  
}
```

Passo 3 — pagamento-service chama o pedido-service

Aqui é onde a comunicação entre microsserviços acontece de fato: uma requisição HTTP de um serviço para o outro.

```
@RestController
@RequestMapping("/pagamentos")
public class PagamentoController {

    private final RestTemplate restTemplate;
    private final String pedidoServiceUrl; // vem do application.properties

    @PostMapping("/processar/{pedidoId}")
    public ResponseEntity<Map<String, Object>>
        processarPagamento(@PathVariable Long pedidoId) {

        String url = pedidoServiceUrl + "/pedidos/" + pedidoId + "/status";

        // chamada HTTP: pagamento-service -> pedido-service
        restTemplate.put(url, new AtualizarStatusRequest("PAGO"));

        return ResponseEntity.ok(Map.of("pagamento", "APROVADO (simulado)"));
    }
}
```

Passo 4 — Testando o fluxo completo

- 1 Suba os dois serviços**
`mvn spring-boot:run` em cada pasta (dois terminais)
- 2 Crie um pedido**
`POST http://localhost:8081/pedidos`
- 3 Processe o pagamento**
`POST http://localhost:8082/pagamentos/processar/1`
- 4 Confirme o status**
`GET http://localhost:8081/pedidos/1` → deve estar "PAGO"

```
$ curl -X POST http://localhost:8082/pagamentos/processar/1
{ "pedidoId": 1, "pagamento": "APROVADO (simulado)", "mensagem": "Status atualizado para PAGO..." }

$ curl http://localhost:8081/pedidos/1
{ "id": 1, "status": "PAGO", ... }
```

O que apareceu na prática hoje?



Transparência de acesso

O pagamento-service chama o pedido-service com uma requisição HTTP simples, igual chamaria qualquer API externa.



Cada serviço, seu próprio dado

Os pedidos vivem só dentro do pedido-service — o pagamento-service não tem (nem deveria ter) acesso direto a eles.



Falha parcial de verdade

Se o pedido-service cair, a chamada do pagamento-service falha — algo que não existe dentro de um monólito.



Openness na prática

Usamos REST/JSON — por isso deu para testar com curl, Postman ou até o navegador, sem ferramenta nenhuma específica.



PARA A PRÓXIMA AULA

Módulo 2: Arquitetura de Microserviços

- Evolução de arquiteturas e princípios de decomposição em serviços
- Padrões e anti-padrões (como reconhecer um "monólito distribuído")
- Vamos evoluir o Mini iFood: melhorar contratos de API e tratar erros de comunicação entre pedido-service e pagamento-service



Código de hoje (pedido-service + pagamento-service) disponível para revisão em casa — usem como referência para continuar praticando.